

AI Prompt Engineering — Writing Sample

Robin Renee Karim

Purpose

This sample demonstrates core prompt engineering techniques and the reasoning behind them: how a prompt is structured, why it's built that way, and how the output is checked before it's used. The techniques shown here are general-purpose — they apply to any role that involves working with AI tools (Claude, ChatGPT, Copilot, or similar), regardless of industry or job function.

Each example includes: - **Technique** — the prompt engineering pattern being demonstrated - **Prompt** — the structured prompt used - **Why it's built this way** — the reasoning behind the structure - **Review step** — how the output was checked before use

Example 1: Role-Based Prompting for Audience-Specific Output

Technique: Assigning the AI a specific role and audience constraint to shape tone, vocabulary, and depth.

Prompt:

You are a communications specialist writing for a non-expert audience with no background in this subject.

Task: Explain [topic] in plain language.

Constraints:

- No jargon; if a technical term is unavoidable, define it in the same sentence
- Maximum 150 words
- End with one practical takeaway the reader can act on

Topic: [insert topic here]

Why it's built this way: - Naming a specific role and audience gives the model a consistent frame of reference, which produces more consistent tone across multiple outputs than a vague instruction like "explain simply." - The word limit and "define jargon inline" rule force concision rather than relying on the model's default verbosity. - Ending on a practical takeaway keeps the output actionable rather than purely descriptive.

Review step: Read the output as a first-time reader would, checking that no assumed background knowledge slipped in, then trimmed anything that restated the question instead of adding new information.

Example 2: Few-Shot Prompting for Consistent Formatting

Technique: Providing 2-3 worked examples so the AI matches an exact format, rather than describing the format in the abstract.

Prompt:

Convert each item below into the following format:

Example 1:

Input: "Meeting moved from Tuesday to Thursday, same time"

Output: {"change_type": "reschedule", "original": "Tuesday", "new": "Thursday", "time_changed": false}

Example 2:

Input: "Budget increased by 10%, deadline unchanged"

Output: {"change_type": "budget", "original": null, "new": "+10%", "time_changed": false}

Now convert:

Input: "[insert new item here]"

Output:

Why it's built this way: - Showing the exact output shape through examples is more reliable than describing a schema in prose — the model pattern-matches to the examples rather than interpreting a written spec. - Including two examples with slightly different structures (one with a null field) shows the model how to handle a case that doesn't perfectly match the first example.

Review step: Ran several real inputs through the prompt and manually checked each output against the intended schema before treating the format as reliable enough for repeated use.

Example 3: Chain-of-Thought Prompting for Multi-Step Reasoning

Technique: Instructing the AI to show its reasoning steps before giving a final answer, which improves accuracy on tasks with multiple dependent steps.

Prompt:

Work through this step by step before giving your final answer.

1. First, list the individual factors relevant to this decision.
2. Second, weigh each factor against the stated priority: [insert priority].
3. Third, note any factor that conflicts with another.
4. Only after those three steps, give a final recommendation in one sentence,
labeled "Recommendation:".

Situation: [insert situation here]

Why it's built this way: - Forcing the model to enumerate factors before concluding reduces the chance it jumps to a plausible-sounding answer without actually weighing the inputs. - Labeling the final line makes the recommendation easy to extract programmatically or by

eye, separate from the reasoning trail. - Asking it to flag conflicts surfaces trade-offs a reader needs to know about, rather than hiding them behind a single confident-sounding answer.

Review step: Checked the listed factors against the actual situation for anything the model invented or omitted, before accepting the final recommendation.

Example 4: Constraint-Based Prompting for Reliability

Technique: Explicitly telling the AI what *not* to do — flag uncertainty instead of guessing — so the output is safe to use in situations where a wrong guess is worse than an honest gap.

Prompt:

Answer using only the information provided below. Do not use outside knowledge and do not fill in gaps with assumptions.

If the provided information doesn't fully answer the question, respond with what IS supported, then add: "Not addressed in the source material: [describe the gap]"

Source information:
[insert source text here]

Question: [insert question here]

Why it's built this way: - Models default to being helpful even when that means guessing — this prompt makes "I don't have that information" an acceptable and expected output rather than a failure state. - Separating what's supported from what's missing keeps the reader from having to guess which parts of the answer are grounded and which aren't.

Review step: Spot-checked several answers against the source material to confirm nothing outside the provided text had leaked into the response.

Example 5: Iterative Refinement Prompting

Technique: Treating the first AI output as a draft, then using a follow-up prompt to refine a specific dimension rather than regenerating from scratch.

Prompt (refinement pass):

Here is a draft: [insert draft].

Revise it with these specific changes only:

- Shorten the second paragraph by about half
- Replace the closing sentence with a direct call to action
- Do not change anything else, including word choice in the first paragraph

Return only the revised version.

Why it's built this way: - Naming exactly what should change (and stating what should NOT change) prevents the common problem of a "revision" quietly rewriting parts that were already fine. - This turns editing into a controlled, repeatable process instead of a full regeneration each time, which matters when earlier parts of a draft have already been approved.

Review step: Compared the revised version line-by-line against the original draft to confirm only the requested sections changed.

Example 6: Using AI to Evaluate AI Output

Technique: A second prompt that checks the first prompt's output against explicit criteria, rather than trusting the first output at face value.

Prompt:

Review the text below against these criteria only. Do not rewrite the text.

For each criterion, respond with Pass, Fail, or Unclear, and a one-sentence reason:

1. Does it directly answer the original question?
2. Is every claim traceable to the source material provided?
3. Is the tone appropriate for the stated audience?
4. Is it free of information that contradicts the source material?

Original question: [insert]

Source material: [insert]

Text to review: [insert]

Why it's built this way: - Separating generation from evaluation into two distinct prompts avoids the model grading its own homework in the same breath it wrote it — a fresh pass with explicit criteria catches more issues. - Pass/Fail/Unclear with a one-line reason keeps the review fast to read and easy to act on, rather than open-ended commentary.

Review step: Any "Fail" or "Unclear" result triggered a manual look at that specific section before the content was finalized; passing this check was treated as a first-pass filter, not a substitute for final human sign-off.

Takeaways for Reviewers

- These six techniques — role-based prompting, few-shot examples, chain-of-thought reasoning, constraint-based reliability, iterative refinement, and AI-assisted evaluation — cover the core patterns used across most practical AI-prompting work, regardless of industry or job title.
- Every example includes a human review step, reflecting a working assumption that AI output is a strong first draft, not a finished, unsupervised deliverable.
- These patterns transfer directly to new contexts: the same structure used for a plain-language explanation above works equally well for a customer email, a research summary, a training document, or a data-formatting task — the technique doesn't

change, only the subject matter does.

This document is part of a writing-samples portfolio. See the full repository for additional samples.